第三届"兴智杯"全国人工智能创新应用大赛

织序 WeaveOrder

可审计的企业流程智能体

项目名称	织序 WeaveOrder —— 可审计的企业流程智能体
主题赛名称	云智专项赛
赛题名称	云上企业级流程智能体创新应用方案赛
团队名称	兰儒恺
填报日期	2026 年 11 月

中国信息通信研究院等 · 主办 | 华为云计算技术有限公司 · 支持

项目基础信息

项目名称	织序 WeaveOrder —— 可审计的企业流程智能体
应用行业及场景	智慧零售 · 连锁零售企业经营分析（销售/利润/折扣/退货归因）
核心技术	自然语言处理（需求理解） · 智能体编排（确定性规划与多工具协同） · 数据工程（跨表 SQL 与确定性校验）
应用发展阶段	应用示范阶段（已在真实公开数据集上完成端到端验证，含完整评测与对照实验）

一、项目内容

1.1 背景及意义

行业痛点：当前企业 AI 应用多停留在文案生成、知识问答等"单点试用"阶段，尚未深度嵌入生产、经营、管理等核心流程。以零售经营分析为例，一线与管理者普遍面临三类困难：

- **系统难连接：**销售、退货、区域负责人等数据分散在多张表中，需要人工跨表关联；
- **流程难闭环：**从"提出问题"到"拿到结论"要经历取数、清洗、统计、核对、成文多个断点；
- **价值难量化：**分析结果是否可信、效率提升多少，缺乏可验证、可审计的口径。

场景价值：本项目以连锁零售经营分析为切入点，把"自然语言提问 → 数据洞察 → 决策建议"的完整链路自动化，并将每一步工具调用、每一个数字都留痕、可复算。

> 实测口径：在同一数据、同一问题下，人工按标准分析作业流程（SOP）需 **8 步、约 70 分钟**；织序智能体端到端 **1.4 秒**完成，见 §2.4 与附录。

1.2 主要场景及功能

核心场景：连锁零售企业经营分析。数据为公开的 Tableau Superstore 零售数据集，含 **9,994 笔真实订单**、21 个字段，以及退货清单（296 单）与区域负责人（4 人）两张辅助表。

主要功能：用户用一句自然语言提出分析需求（如"找出亏损的子品类并归因"），系统自动完成五段闭环：



- **信息理解：**由大模型把自然语言解析为结构化的分析要点（意图、焦点、维度、指标），实测可正确识别"亏损归因 / 趋势 / 区域 / 品类概览"四类意图；
- **流程判断：**按意图生成确定性的分析 DAG（有向无环图），规划结果可复现、可审计；

- **系统调用：**通过工具注册表调用数据服务（只读 SQL 真执行）与报表服务，同层步骤并行；
- **结果输出：**报告中的指标与表格全部来自 SQL 结果，由校验器独立重算比对，不一致即拦截、不发布。

1.3 实现方案

设计理念：理解靠大模型，规划用确定性模板。大模型负责"听懂需求"，规划层负责"保证可复现"，两者职责分离，避免模型自由发挥导致的结果漂移。

系统架构（五层）：

- **接入层：**CLI / Web 控制台 / A2A 协议（Agent-to-Agent）多入口接入；
- **理解层：**大模型信息抽取，输出结构化分析要点，异常时回退规则解析保证离线可用；
- **规划层：**按意图选择分析模板，生成 DAG（步骤、依赖、工具、参数）；
- **执行层：**工具注册表统一校验参数（JSON Schema）、风险分级、超时控制，同层并行执行；
- **校验层：**确定性校验器独立重算每个报告指标，并核对数据源一致性。

系统对接与数据贯通：本方案对接三类系统/工具，满足"接得上、连得通"：

系统/工具	作用	调用与留痕方式
数据服务（SQLite 引擎）	装载 orders / returns / people 三张表，执行只读 SELECT	5 次 sql_query 调用，SQL 与结果全量留痕
报表服务	从 SQL 结果解析指标引用、生成 Markdown 报告与明细表	1 次 report_render 调用
大模型推理服务	自然语言需求理解（结构化信息抽取）	1 次调用，消耗 345 tokens

数据流转：三张数据表装载入库后，由 DAG 中的多条 SQL 分别完成"子品类盈亏、折扣档位、跨表退货率、品类汇总、整体指标"五类聚合；报告指标通过引用解析从 SQL 结果中取值，**不手抄数字**，校验器再用同一套查找逻辑独立重算比对。

1.4 项目创新点

- **确定性校验器（质量保障机制化）：**报告中的每个指标都由校验器从数据源独立重算，不一致即拦截。同一注入实验中，校验开时错误数字被当场拦截，校验关时错误数字直接发布（详见 §2.4），把"人工交叉核对"变成系统能力。
- **跨表数据贯通：**退货率分析将 orders 与 returns 通过 order_id 做 LEFT JOIN，实现跨系统/跨表的数据贯通，而非单表统计。
- **可复现的编排：**规划层采用确定性模板，同一需求任意次运行得到一致的分析路径与结果，便于审计与回溯。
- **跨场景可复用：**引擎原语（工具注册表、事件溯源、确定性校验、产物布局）与场景解耦，只替换工具集与数据源即可迁移——已在差旅编排、经营分析、零售分析三类场景验证。

二、应用分析

2.1 市场分析

企业 AI 落地的主战场正从"单点辅助"转向"流程智能"。据信通院等机构对传统企业数字化转型的调研，经营分析、报表生成、流程自动化是需求最集中的三类场景，其共同特征是**高频、重复、规则相对清晰**，最适合由智能体承接。

本方案面向连锁零售、电商、区域经销等拥有多表经营数据的企业，提供"用一句话完成一次经营分析"的能力。相较市面常见的 BI 工具，本方案无需用户编写 SQL 或拖拽报表，且每个数字都可回溯到数据源，适合快速推广。

2.2 商业模式分析

- **市场定位：**面向中型连锁零售企业的"分析流程自动化"组件，可嵌入既有数据平台；
- **交付方式：**以引擎 + 场景模板的形式提供，按行业场景配置工具集；
- **定价参考：**按分析任务量或订阅制收费，边际成本主要为模型调用成本（实测单次约 345 tokens）；
- **竞争壁垒：**确定性校验与全链路留痕，解决"AI 报告数字不可信"的核心顾虑。

2.3 应用情况

本方案已在真实公开数据集上完成端到端验证，并形成完整的评测与对照实验：

指标	实测结果
数据集	Tableau Superstore 公开零售数据：9,994 笔订单 / 296 笔退货 / 3 张表
端到端耗时	**1.4 秒**（含 1 次大模型调用）
分析步骤	7 步全部成功（7 次工具调用，含 1 次大模型信息理解）
Token 消耗	345 tokens（prompt 240 + completion 105）
评测通过率	**7/7**（含意图识别、黄金值核验、注入拦截、消融对照）
黄金值来源	评测脚本从 CSV 独立重算，与被测实现零共享代码

2.4 实际应用效果：人工基线 vs 智能体

对照任务：**分析 Superstore 零售数据，找出亏损的子品类并归因**（同一数据、同一问题）。

人工基线按标准分析作业流程（SOP）逐步记录：

#	步骤	操作内容	标准工时
1	数据理解	识别 3 张表、21 个字段、9,994 行订单	5 分钟
2	建库导入	创建分析库、导入 orders 与 returns	5 分钟
3	子品类汇总	编写并调试按子品类的销售/利润汇总 SQL	10 分钟
4	识别亏损	排序并核对亏损子品类	5 分钟
5	折扣分档	编写并调试折扣档位盈亏 SQL	10 分钟
6	指标计算	手工计算利润率并交叉核对	10 分钟
7	跨表关联	建立退货表并编写 LEFT JOIN 计算退货率	10 分钟
8	报告撰写	整理数字、撰写发现与结论	15 分钟
合计			**约 70 分钟**

量化对比：

维度	人工基线	织序智能体	提升
端到端耗时	约 70 分钟	1.4 秒	**约 3,000×**
操作步骤	8 步手工（含 4 段 SQL 编写调试）	7 步自动	—
数字准确性	依赖人工交叉核对，易出错	校验器独立重算，错即拦截	质量保障机制化
可复现性	依赖个人经验与习惯	确定性规划，结果一致	可审计、可复现
单次成本	分析师时间成本（约 70 分钟）	345 tokens	边际成本趋近于零

质量对照（同一错误注入实验）：对报告关键指标注入同样的错误（把利润率算错一半）——校验器开启时，报告被拦截、不发布；校验器关闭时，错误数字直接发布（+6.2% vs 正确值 +12.5%）。人工流程若要达到同等保障，需额外增加交叉核对步骤（约 +10 分钟）。

2.5 社会效益

- **降低分析门槛：**业务人员无需掌握 SQL 或 BI 工具，用自然语言即可获得可信的分析结论；
- **提升决策效率：**把经营分析从"小时级"压缩到"秒级"，让分析能力随业务规模弹性扩展；
- **沉淀可复用能力：**引擎与场景解耦，新的行业场景只需配置工具集与数据模板，缩短交付周期；
- **可信 AI 实践：**全链路留痕 + 独立校验，为企业级智能体的合规与审计提供可参考的实现范式。

附录

附录 1 · 运行档案（系统交互截图）

下图为本次申报材料的运行档案页，完整呈现"任务输入 → 信息理解 → 流程判断 → 系统调用 → 结果输出"的闭环，以及每一步的工具调用与校验记录。



织序 WeaveOrder · 运行档案

运行 ID：20261103-104656-208-rt · 生成时间：2026-11-03T11:55:01

任务输入

自然语言分析需求

分析 Superstore 零售数据，找出亏损的子品类并归因

信息理解（大模型）

识别亏损的子品类并分析其亏损原因

意图：profit_diagnosis　来源：llm

维度：sub_category、discount、region　指标：profit、sales、margin

流程判断（确定性规划）

任务输入 → 信息理解 → 流程判断 → 系统调用 → 结果输出

步骤	目标	工具	依赖
s1	读取零售数据集（orders/returns/people 三张表画像）	file_read	—
s2	子品类盈亏榜（按利润升序）	sql_query	s1
s3	折扣档位盈亏（定位亏损是否由折扣驱动）	sql_query	s1
s4	退货率最高的子品类（跨表 JOIN orders×returns）	sql_query	s1
s5	品类汇总	sql_query	s1
s6	整体汇总指标	sql_query	s1
s7	生成经营分析报告（指标全部引用 SQL 结果）	report_render	s2, s3, s4, s5, s6

系统调用（工具注册表 · 真执行）

#	步骤	工具	参数	时刻
5	s1	file_read	–	1272 ms
13	s2	sql_query	SELECT sub_category, ROUND(SUM(sales),2) AS sales, ROUND(SUM(profit),2…	1374 ms
14	s3	sql_query	SELECT CASE WHEN discount=0 THEN '0%' WHEN discount<=0.2 THEN '1-20%' …	1374 ms
15	s4	sql_query	SELECT o.sub_category AS sub_category, ROUND(COUNT(DISTINCT r.order_id…	1374 ms
16	s5	sql_query	SELECT category, ROUND(SUM(sales),2) AS sales, ROUND(SUM(profit),2) AS…	1374 ms
20	s6	sql_query	SELECT ROUND(SUM(sales),2) AS total_sales, ROUND(SUM(profit),2) AS tot…	1377 ms
30	s7	report_render	–	1387 ms

通过 确定性校验器

全部指标与数据源一致

运行指标

1387 ms	7	7	345	ok
端到端耗时	分析步骤	工具调用	Tokens 消耗	最终状态

图 1a　运行档案（上）：任务输入、信息理解、流程判断与分析 DAG、系统调用明细、确定性校验与运行指标

结果输出

Superstore 经营分析 · 亏损归因

- **整体利润率**：+12.5%
- **整体销售额**：2,297,201
- **整体利润**：286,397
- **亏损子品类数**：3
- **退货订单数**：296
- **Tables 利润率**：−8.6%
- **折扣>30% 利润率**：−48.2%
- **退货率最高子品类退货率**：+11.6%

子品类盈亏榜（按利润升序）

sub_category	sales	profit	margin	orders
Tables	206965.53	−17725.48	−8.56	319
Bookcases	114880.0	−3472.56	−3.02	228
Supplies	46673.54	−1189.1	−2.55	190
Fasteners	3024.28	949.52	31.4	217
Machines	189238.63	3384.76	1.79	115
Labels	12486.31	5546.25	44.42	364
Art	27118.79	6527.79	24.07	796
Envelopes	16476.4	6964.18	42.27	254
Furnishings	91705.16	13059.14	14.24	957
Appliances	107532.16	18138.01	16.87	466
Storage	223843.61	21278.83	9.51	846
Chairs	328449.1	26590.17	8.1	617
Binders	203412.73	30221.76	14.86	1523
Paper	78479.21	34053.57	43.39	1370
Accessories	167380.32	41936.64	25.05	775
Phones	330007.05	44515.73	13.49	889
Copiers	149528.03	55617.82	37.2	68

折扣档位盈亏

discount_band	orders	sales	profit	margin
0%	4798	1087908.47	320987.6	29.51
1–20%	3803	846522.24	100785.47	11.91
21–30%	227	103226.65	−10369.28	−10.05
>30%	1166	259543.49	−125006.78	−48.16

品类汇总

category	sales	profit	margin
Technology	836154.03	145454.95	17.4
Office Supplies	719047.03	122490.8	17.04
Furniture	741999.8	18451.27	2.49

发现

- 部分子品类整体亏损（Tables、Bookcases、Supplies），是利润的主要漏点
- 折扣越高亏损越严重：折扣 ≥30% 的订单利润率为负，属于「以折扣换规模」的亏损区
- 退货集中在少数子品类，需结合退货原因与物流数据进一步排查

结论

亏损由「高折扣 + 特定子品类」共同驱动：建议收缩 ≥30% 的折扣档位，并针对 Tables 等亏损子品类重谈采购价或调整定价。

本档案由 runs/20261103-104656-208-rt/ 的 trace/plan/result 自动生成，数字均可回溯。

图 1b　运行档案（下）：结果输出 —— 数据驱动报告快照（关键指标、明细表与结论）

附录 2 · 演示视频（扫码观看）

https://raw.githubusercontent.com/iakRulan/weaveorder-agent/main/docs/177-demo.mp4

图 2　演示视频（时长 1 分 19 秒，含真实运行与对照实验）

附录 3 · 评测报告（黄金值独立重算）

评测共 7 个用例，覆盖意图识别、主用例指标核验、注入拦截与消融对照，全部通过。黄金值由评测脚本直接从 CSV 独立重算，与被测实现零共享代码。

Task	验证内容	结果
R01	主用例：亏损归因（Tables 利润率 -8.6%、折扣>30% 利润率 -48.2%、整体利润率 +12.5%）	PASS
R02	意图识别：趋势（trend）	PASS
R03	意图识别：区域（region），关联 people 表负责人	PASS
R04	意图识别：品类概览（category_overview）	PASS
R05	注入错误 + 校验开 → 拦截，不发布	PASS
R06	注入错误 + 校验关 → 错误数字直接发布（消融对照）	PASS
R07	跨表退货率：orders × returns JOIN（最高 Machines 11.6%）	PASS

黄金值（评测脚本独立重算）：Tables 利润率 **-8.56%**；亏损子品类 **3** 个；折扣>30% 利润率 **-48.16%**；整体利润率 **12.47%**（销售额 2,297,201）；退货订单 **296** 单，退货率最高 **Machines 11.61%**。

附录 4 · 证据材料清单

- **运行档案：**任务输入、意图识别、分析 DAG、工具调用、校验记录、最终报告（图 1）；
- **API 调用日志：**7 次工具调用的时间、参数与结果，以及完整事件序列（34 条）；
- **数据流转记录：**三张数据表 → SQL 聚合 → 报告指标的完整链路，含跨表 JOIN 说明；
- **人工基线对比：**8 步标准作业流程的操作内容与工时，及与智能体的量化对照；
- **代码与数据：**分析场景运行器、工具层、数据预处理脚本与评测脚本（核心摘录见附录 5）。

附录 5 · 核心代码摘录

（1）流程判断：按分析意图生成确定性分析 DAG（节选自 retail_agent.py）。

```
def build_plan(brief):
    """按分析意图生成确定性的分析 DAG（可复现、可审计）。"""
    intent = brief.get("intent") or "general"
    steps = [{"id": "s1", "tool": "file_read", "args": {}, "depends_on": []}]
    if intent == "profit_diagnosis":
        steps += [
            {"id": "s2", "tool": "sql_query",
             "args": {"sql": SUB_CATEGORY_PROFIT}, "depends_on": ["s1"]},
            {"id": "s3", "tool": "sql_query",
             "args": {"sql": DISCOUNT_BANDS},   "depends_on": ["s1"]},
            {"id": "s4", "tool": "sql_query",
             "args": {"sql": RETURN_TOP},       "depends_on": ["s1"]},
            {"id": "s6", "tool": "sql_query",
             "args": {"sql": OVERVIEW},         "depends_on": ["s1"]},
        ]
        steps.append({"id": "s7", "tool": "report_render",
                      "args": {"metrics": METRICS, "tables": TABLES},
                      "depends_on": ["s2", "s3", "s4", "s6"]})
    return {"goal": brief["focus"], "intent": intent,
            "strategy": "dag", "created_by": "deterministic-planner",
            "steps": steps}
```

（2）质量保障：校验器独立重算每个报告指标，不一致即拦截（节选自 retail_agent.py）。

```
def _verify_report(self, out):
    """独立重算：不走渲染路径，直接从 SQL 结果重新解析每个指标。"""
    for metric in out.get("metrics") or []:
        name = metric.get("name", "")
        expected = metric.get("current")
        if metric.get("baseline") is not None:
            base = metric.get("baseline")
            expected = ((expected - base) / base * 100
                        if metric.get("unit") == "%" else expected - base)
        if abs(float(metric.get("value", 0)) - float(expected)) > 0.01:
            return False, (f"「{name}」报告值 {metric.get('value')} 与数据源"
                           f"重算值 {round(float(expected), 4)} 不一致，已拦截")
    return True, "全部指标与数据源一致"
```

（3）系统调用：工具注册表统一校验参数、风险分级与超时控制（节选自 core/tools.py / core/retail.py）。

```
registry = ToolRegistry()
registry.register(Tool(
    name="sql_query",
    description="对零售数据执行只读 SELECT 查询（支持 orders/returns/people 跨表 JOIN）",
    parameters={"type": "object",
                "properties": {"sql": {"type": "string"}},
                "required": ["sql"]},
    fn=lambda **kw: sql_query(state, **kw),
    risk=0, category="retail",
    returns="sql/columns/rows/row_count",
))
# 调用：参数先经 JSON Schema 校验/纠偏，再执行；全过程写入 trace.jsonl
```

> 声明：本材料中的数据来自公开的 Tableau Superstore 零售数据集，仅用于竞赛演示与研究； 所有运行指标、调用日志与对比数据均来自真实运行产物，可复现、可审计。